

- Steps required for inserting files: n (1 step for inserting each of n files)

If we add these up, we get an expression like the following:

$$\text{maximum_total_steps} = n + (1 + 2 + \dots + n)$$

We can simplify this expression using a handy identity:

$$1 + 2 + \dots + n = \frac{n \times (n + 1)}{2}$$

If we use this identity and then add everything together and simplify, we find that the total number of steps required is

$$\text{maximum_total_steps} = \frac{n^2}{2} + \frac{3n}{2}$$

We finally have a very precise expression for the maximum total steps that could be required to perform insertion sort. But believe it or not, this expression may even be too precise, for several reasons. One is that it's the maximum number of steps required, but the minimum and the average could be much lower, and almost every conceivable list that we might want to sort would require fewer steps. Remember the jaggedness in the curve we plotted in [Figure 4-1](#)—there's always variation in how long it takes to perform an algorithm, depending on our choice of input.

Another reason that our expression for the maximum steps could be called too precise is that knowing the number of steps for an algorithm is most important for large values of n , but as n gets very large, a small part of the expression starts to dominate the rest in importance because of the sharply diverging growth rates of different functions.

Consider the expression $n^2 + n$. It is a sum of two terms: an n^2 term, and an n term. When $n = 10$, $n^2 + n$ is 110, which is 10% higher than n^2 . When $n = 100$, $n^2 + n$ is 10,100, which is only 1% higher than n^2 . As n grows, the n^2 term in the expression becomes more important than the n term because quadratic functions grow so much faster than linear ones. So if we have one algorithm that takes $n^2 + n$ steps to perform and another algorithm that takes